
Traitement du Signal Temps Réel

Sujet BE: Autotune

Olivier Perrotin & Thomas Hueber

DURATION: 4×4 h
DATE DE RENDU : 09 Janvier 2025

Introduction

1 Purpose of the BE

The lecture (2×2 h) covered various aspects related to the design of a real-time system:

- Definitions of a real-time system ($\neq$ fast execution system)
- Theoretical design models (synchronous/scheduled, etc.)
- Hardware selection (DSP, GPU, etc.)
- Software implementation techniques (best practices, priority inversion risk, etc.)
- Audio on PC (multi-layered software, audio APIs, etc.) and real-time audio processing (producer-consumer model, buffer underrun/overflow, overlap-add, circular buffer).

A particular focus was placed on “soft” systems, intended for implementation on a standard OS. As part of the TSTR project (4×4 h), you will put into practice some of the points covered in the lecture through the **implementation of an *autotune* audio effect using additive synthesis**.

2 Evaluation

You will demonstrate your program in 5 min during the second part of the 4th session.

You will provide a ZIP file named `name1_name2_tstr_2025.zip`, to be sent to `olivier.perrotin@grenoble-inp.fr`, containing:

- A README specifying the steps to compile and run your program.
- Your `duplex.cpp` code, cleaned and commented for each question in the project.
- A `files/` directory containing the recordings, with each file named explicitly and properly referenced in the report (the name must include the original file name, the question number, and the data type (e.g., in, out, f_0). Audio recordings must be provided in wav format, not binary.
- A *short* report (max 4 pages, named `name1_name2_tstr_2025.pdf`) detailing your implementation choices and the results obtained. Instructions for which a response is expected in the report (answer to a question, plotting a figure, etc.) are indicated in **blue** in the following instructions. You are encouraged to also indicate any issues encountered.

This project will be implemented in standard C under Linux, **on school machines**, using the RtAudio API developed by G. Paul Scavone.

In your report, pay particular attention to *spelling*, as well as *figures* (explicit axis titles, readable text, effective use of colours, etc.).

3 Resources

The project resources are in the `BE_tstr_2025_v1` folder on Chamilo. You will find:

- The RtAudio API `rtaudio-6.0.1.tar`
- A `c/` directory containing C functions that will be useful to you, described in Appendix 2.
- A `python/` directory containing a Python script to plot signals recorded in binary format by your program.
- An `audio_files` directory containing audio files to test as input to your system.

Source and documentation of RtAudio: <https://www.music.mcgill.ca/~gary/rtaudio/>

Getting Started with the RtAudio API

1 Basic API Functionality

1. Compile RtAudio on Linux by following these steps:
 - a) Download the RtAudio API (`rtaudio-6.0.1.tar`) and extract it to your chosen directory.
 - b) Activate the conda environment `rtaudio` (see Appendix 3 if issues arise).
 - c) Compile the main library (`./configure` followed by `make all`).
 - d) Compile the example programs (`cd tests` followed by `make all`).
2. First, run `audioprobe` and then `duplex`. [What do these two programs do?](#)
3. Study the source code `duplex.cpp`, which manages an audio I/O stream:
 - a) Identify the audio stream configuration parameters (sampling rate, number of internal buffers, buffer size, etc.), and the `openStream()` and `inout()` functions. [What are the roles of these two functions? Identify the arguments of `openStream\(\)` and `inout\(\)` for exchanging data between main and audio callback.](#)
 - b) Three of the arguments of the `inout()` function are of type `void`. [Explain why, and explain the method to access these arguments \(line 84 of the original code\).](#) Apply this method to the input/output buffers and replace the `memcpy()` line with a loop that copies the input buffer to the output buffer element by element.
 - c) Instead of passing a simple variable in the `userData` argument of `openStream()` (currently the buffer size in bytes), it is possible to pass a data structure containing as many variables as you want. Implement this solution.
 - d) For the rest of the project, we will work with doubles. Configure the RtAudio API to work with this format (see the `RTAUDIO_FLOAT64` flag).

In this project, you will implement a real-time C/C++ version of autotune starting from the `duplex.cpp` source code. In the `c/somefunc_2025.cpp` file of the `BE_tstr_2025_v1` archive, you will find a set of functions designed to facilitate your implementation, listed in Appendix 2. To simplify compilation and use the existing Makefile, you will only work in `duplex.cpp`, adding the necessary functions and their prototypes.

Debugging Toolbox

2 Reading a file – Testing specific inputs

To test your program with controlled inputs and reproduce errors, it is good practice to use pre-recorded audio files as system inputs. A set of test files is provided in the `fichiers_audio_test/` directory of the `BE_tstr_2025_v1` archive, both in `.wav` format for easy listening on your machine, and in binary format (`.bin`, headerless) in double format (64 bits), and thus easily readable with `fread()`.

4. Load one of the files using Algorithm 1 in Appendix 1. Useful functions: `fopen()`, `fread()`, `fseek()`, `ftell()`, `fclose()`. [Where do you place the loading of the audio file in the program, and why?](#)
5. Now, simulate a real-time input where, at each call of the `inout()` callback function, you copy the next N samples from the audio file into the input buffer of size N . By keeping the copy from the input buffer to the output buffer, you should then hear the playback of your file when calling `duplex`.
6. Adapt your code so that the audio file is played in a loop.

3 Writing a file – Observing variable evolution

There are few simple tools in C that allow you to observe and plot the evolution of variables over time. One solution is to write the variables to a file during execution and then plot them with an external program. The `write_buff_dump()` function provided to you (see Appendix 2.1) allows you to gradually fill a buffer called `buffer_dump` with the variable or array of your choice during the execution of the program.

7. Initialize a `buffer_dump` array in your program, with the size of your choice. Useful functions: `malloc()`, `calloc()`, `free()`. [Do you initialize it in `main\(\)` or `inout\(\)` and why?](#)
8. Copy the input buffer from the `inout()` callback into the `buffer_dump` buffer as the program executes using `write_buff_dump()`.
9. Write the `buffer_dump` buffer to disk in a binary file. Useful functions: `fopen()`, `fwrite()`, `fclose()`. [Do you write it in `main\(\)` or `inout\(\)` and why?](#)
10. The `python/plot_dump.py` script allows you to read the written binary file and plot the data. Adapt the filename to be read in the script and customize the figure display as needed. This script also writes the signal to disk in `.wav` format to allow you to listen to it, in the case where the recorded signal is audio. To use `matplotlib`, use the conda environment `pytorch`.
11. Repeat the procedure with a second buffer to also write the output signal from the `inout()` function.

For each run of the program, the input and output of the program are now written to files.

- In the rest of the project, you will be regularly asked to include these files to verify your implementation. You will also sometimes be asked to write intermediate variables from your program to observe them.
- Beyond the instructions, you are also strongly encouraged to write and observe the variables in your program that you deem necessary to help you debug your code.

Autotune audio effect using additive synthesis

4 Signal analysis

Fundamental frequency: In the lecture, we saw that a method for estimating f_0 uses the second maximum of the autocorrelation of the signal to be analysed.

12. Implement the autocorrelation function on the input buffer of `inout()` and extract the second maximum to obtain f_0 . You can use Algorithm 2 provided in Appendix 1. Use the various test audio files provided and plot the measured f_0 over time (in seconds) with `plot_dump.py` after writing it to a file. [Include in your report your plots of \$f_0\$ as well as the files containing the corresponding input signals. Comment on your results.](#)
13. To reduce detection errors, limit the search for the fundamental frequency to a range corresponding to the vocal range of your choice.
14. Test the f_0 analysis for several buffer sizes. [Provide the corresponding plots and comment. Why would a circular buffer be useful here?](#) You will implement the circular buffer at the end of the project. In the meantime, choose the buffer size you find most suitable.

Harmonic analysis: To perform additive synthesis, it is necessary to know the frequencies and amplitudes of the harmonics to be generated.

15. Let `n_fft` be the number of points on which you calculate a Discrete Fourier Transform (DFT). [What is the value of the DFT bin closest to the measured \$f_0\$?](#)
16. Calculate the DFT of the signal (see Appendix 2.4) and extract the frequency and amplitude values for each harmonic. Note that an FFT gives you the DFT up to a factor of `n_fft`. (For example, applying the FFT to a cosine gives two peaks of amplitude `n_fft/2`). Normalize the FFT output accordingly to obtain the actual amplitude of the DFT.

5 Signal synthesis

We will now write a new signal to the output buffer.

By default, the values -1 and 1 of the output buffer correspond to the maximum amplitude level in absolute value of your sound card. To protect your ears:

- Ensure that you do not send values beyond this range to the output buffer.
- The amplitude `[-1,1]` can be very loud depending on your audio equipment. On the first run of your program, wait to hear the sound level before putting the headphones on your ears.

First implementation: Additive synthesis generates a signal by adding cosines with amplitudes and frequencies corresponding to each harmonic of the signal to be reconstructed (see the equation in the lecture).

17. Implement additive synthesis from the frequencies and amplitudes of the previously identified harmonics and send the result to the output buffer. You should normally hear many clicks in the signal. [Write the output signal to a file, plot it, and include it in the report. By observing the signal, how do you explain the clicks heard?](#)

Adding phase: The phase information makes contiguous signal frames coherent.

18. Include the phase in the additive synthesis (see the equation in the lecture material). Write the output signal to a file and include it in the report. Then plot this output signal and comment on it. Are the transitions between each frame perfect?

6 Autotune

Between analysis and synthesis, it is now possible to modify the value of f_0 (and the harmonics) to achieve the desired effect. Autotune consists of rounding f_0 to discrete values on a scale in semitones (Western musical scale). A semitone corresponds to $1/12$ of an octave on a logarithmic scale. The conversion of f_0 from Hz to semitones (ST) and vice versa is done by:

$$f_{0ST} = 12 \log_2(f_{0Hz}) \quad (6.1)$$

$$f_{0Hz} = 2^{\frac{f_{0ST}}{12}} \quad (6.2)$$

19. Implement an autotune by converting f_0 to ST, rounding the value, and converting it back to Hz. For stronger effects, round to 3, 4, or 6 semitones.

7 Sliding windows

Analysis: As seen in question 14, implementing a circular buffer allows for measuring a smoothed f_0 while maintaining a reduced input buffer size.

20. Implement a circular buffer on which you will perform the f_0 estimation. Plot the f_0 curve obtained with different buffer sizes and comment. Include the file containing the corresponding input signal each time.

Synthesis: The *overlap-add* technique completely removes discontinuities between two contiguous output frames.

21. Apply a Hanning window (see Appendix 2.3) to your synthesis buffer and implement an *overlap-add* to fill the output buffer.

You will perform your final demonstration with a file of your choice as input, as well as with microphone input.

Appendices

1 Some algorithms

Algorithm 1: Read a binary file

```
// Open a file
1 Open with fopen()

// Calculate its size
2 Move the file pointer to the end of the file with fseek()
3 Read the file pointer position with ftell()
4 Move the file pointer back to the beginning of the file with fseek()

// Read a file
5 Read with fread()
```

Algorithm 2: Half auto-corrélation (positive indices)

```
// For each element of the correlation (positive indices)
1 for  $n = 0 ; n < N_y ; n++$  do
2    $y[n] = 0;$ 
   // Compute the correlation
3   for  $k = n ; k < N_x ; k++$  do
4      $y[n] = y[n] + x[k] \times x[k - n]$ 
5   end
6 end
```

2 Provided functions

In the file `c/somefunc_2025.cpp` of the `BE_tstr_2025_v1` archive.

2.1 Write to a buffer

```
int write_buff_dump(double* buff, const int n_buff, double* buff_dump, const int n_buff_dump, int* ind_dump);
```

Copies the array `buff` of size `n_buff` into `buff_dump` of size `n_buff_dump`. The copy starts at the index `ind_dump` of the array `buff_dump`, which is incremented as the copy progresses. When `ind_dump` reaches `n_buff_dump`, no more data can be written to `buff_dump`.

`double* buff` : Array to be copied.

`const int n_buff` : Number of elements in the array to be copied (if `n_buff = 1`, this is equivalent to passing the variable `buff` by address).

`double* buff_dump` : Array that receives the copy.

`const int n_buff_dump` : Number of elements in the array that receives the copy.

`int* ind_dump` : Write head of the array that receives the copy. This is *passed by address* to be updated during the function call.

2.2 Measure Execution Time

```
double get_process_time();
```

Returns the system clock time.

2.3 Apply a Hanning window to a signal

```
static double *hanning(double *w, const int leng);
```

Fills the array `w` with a Hanning window of size `leng`. Before the function call, `w` is an empty array. After execution, it contains the window.

2.4 Calculate a Fourier Transform using the FFT algorithm

```
int get_nextpow2(int n);
```

Returns the smallest power of 2 greater than or equal to `n`.

```
int fftr(double *x, double *y, const int m);
```

```
int fft(double *x, double *y, const int m);
```

Calculates the Discrete Fourier Transform of a real signal (`fftr`) or a complex signal (`fft`) using the FFT algorithm.

`double *x` : The real part of the signal. These values will be replaced by the real part of the Fourier transform after execution.

`double *y` : The imaginary part of the signal. These values will be replaced by the imaginary part of the Fourier transform after execution.

`const int m` : Size of the DFT.

```
int ifft(double *x, double *y, const int m);
```

Calculates the inverse Discrete Fourier Transform of a signal using the FFT algorithm.

`double *x` : The real part of the Fourier transform. These values will be replaced by the real part of the signal after execution.

`double *y` : The imaginary part of the Fourier transform. These values will be replaced by the imaginary part of the signal after execution.

`const int m` : Size of the DFT.

3 Debug Conda

If `conda activate rtaudio` does not work:

```
conda init
conda activate rtaudio
```

If `conda init` does not work:

```
source ~/.bashrc (to force the use of bashrc)
conda init
conda activate rtaudio
```